

Язык программирования С

Лекция 4: динамическая память (куча), структуры

Лисица Никита

2026

Напоминание: три вида памяти

- **Стек:** выделяется на старте программы, размер предопределён (стек фреймы “выделяет” на стеке компилятор)

Напоминание: три вида памяти

- **Стек:** выделяется на старте программы, размер предопределён (стек фреймы “выделяет” на стеке компилятор)
- **Глобальная память:** выделяется на старте программы, размер известен на момент компиляции (память, нужная под все глобальные переменные)

Напоминание: три вида памяти

- **Стек:** выделяется на старте программы, размер предопределён (стек фреймы “выделяет” на стеке компилятор)
- **Глобальная память:** выделяется на старте программы, размер известен на момент компиляции (память, нужная под все глобальные переменные)
- **Динамическая память (куча):** выделяется программистом, размер произвольный

Динамическая память (куча)

- `void* malloc(size_t size);` – выделяет непрерывный кусок памяти размером `size` байт

Динамическая память (куча)

- `void* malloc(size_t size);` – выделяет непрерывный кусок памяти размером `size` байт
- `void free(void *ptr);` – возвращает память системе для переиспользования

Динамическая память (куча)

- `void* malloc(size_t size);` – выделяет непрерывный кусок памяти размером `size` байт
- `void free(void *ptr);` – возвращает память системе для переиспользования
- Если не удалось выделить память (напр. закончилась свободная память в системе), `malloc` возвращает *нулевой указатель*: `NULL`

Динамическая память (куча)

```
// Выделить память под миллион объектов типа int
int *p = malloc(1000000 * sizeof(int));

// То же самое: if (!p)
if (p == NULL) {
    // Обработаем нехватку памяти
}

// Можем использовать p как указатель на массив:
for (size_t i = 0; i < 1000000; ++i) {
    p[i] = i * i;
}

// Освобождаем память
free(p);
```


Динамическая память (куча)

- Размер выделяемой памяти может быть полностью динамическим (введён пользователем, вычислен на основе других данных, и т.п.) – невозможно для стека и глобальных переменных

```
size_t size;  
scanf("%zu", &size);  
  
char *p = malloc(size);  
  
// Как-то используем p...  
  
free(p);
```

- Как правило, маленькие объекты нет смысла хранить в куче
 - их дешевле скопировать

Динамическая память (куча)

- Как правило, маленькие объекты нет смысла хранить в куче – их дешевле скопировать
- `int *p = malloc(sizeof(int));` – тратим 4 байта на сам `int` (в куче) и 8 байт на указатель (на стеке)

Динамическая память (куча)

- Как правило, маленькие объекты нет смысла хранить в куче – их дешевле скопировать
- `int *p = malloc(sizeof(int));` – тратим 4 байта на сам `int` (в куче) и 8 байт на указатель (на стеке)
 - И ещё 8-16 байт в куче для метаданных аллокатора :)

Динамическая память (куча)

- Как правило, маленькие объекты нет смысла хранить в куче – их дешевле скопировать
- `int *p = malloc(sizeof(int));` – тратим 4 байта на сам `int` (в куче) и 8 байт на указатель (на стеке)
 - И ещё 8-16 байт в куче для метаданных аллокатора :)
- С и С++ позволяют явно управлять расположением объекта в памяти – этим стоит пользоваться :)

```
int foo(size_t size) {  
    int *p = malloc(size);  
    // Что-то посчитали  
    // Забыли вызвать free  
    return p[0];  
}
```

```
int *p = malloc(size);  
// Что-то посчитали  
// Забыли вызвать free  
  
// Перевыделили память  
p = malloc(new_size);
```

- *Утечка памяти*: мы “потеряли” значение указателя на выделенную область $\implies$ не можем её освободить (нам не от чего вызвать `free`)

- При завершении программы память автоматически вернётся системе, зачем её освобождать?

- При завершении программы память автоматически вернётся системе, зачем её освобождать?
- Многие программы работают продолжительное время (браузер, сервер, игра), и память действительно может закончиться

- При завершении программы память автоматически вернётся системе, зачем её освобождать?
- Многие программы работают продолжительное время (браузер, сервер, игра), и память действительно может закончиться
- Код, который вы пишете для “одноразовой” программы, потом могут переиспользовать для другого проекта

- При завершении программы память автоматически вернётся системе, зачем её освобождать?
- Многие программы работают продолжительное время (браузер, сервер, игра), и память действительно может закончиться
- Код, который вы пишете для “одноразовой” программы, потом могут переиспользовать для другого проекта
- Для поиска утечек есть много инструментов: **Valgrind**, **Address Sanitizer**

- `malloc/free` реализованы в стандартной библиотеке C

Динамическая память (куча)

- `malloc/free` реализованы в стандартной библиотеке C
- Они используют специальные потокобезопасные структуры данных, чтобы
 - Выделять новые участки памяти
 - Переиспользовать старые
 - Выполнять дефрагментацию (склеивать соседние неиспользуемые участки)

Динамическая память (куча)

- `malloc/free` реализованы в стандартной библиотеке C
- Они используют специальные потокобезопасные структуры данных, чтобы
 - Выделять новые участки памяти
 - Переиспользовать старые
 - Выполнять дефрагментацию (склеивать соседние неиспользуемые участки)
- $\implies$ Выделение динамической памяти занимает некоторое время (сотни наносекунд / десятки микросекунд)

Динамическая память (куча)

- `malloc/free` реализованы в стандартной библиотеке C
- Они используют специальные потокобезопасные структуры данных, чтобы
 - Выделять новые участки памяти
 - Переиспользовать старые
 - Выполнять дефрагментацию (склеивать соседние неиспользуемые участки)
- $\implies$ Выделение динамической памяти занимает некоторое время (сотни наносекунд / десятки микросекунд)
- $\implies$ Лучше выделять и переиспользовать большой кусок памяти, чем выделять много маленьких объектов

Двумерный массив на куче: вариант 1

```
int **m = malloc(N * sizeof(int*));  
for (int i = 0; i < N; ++i)  
    m[i] = malloc(N * sizeof(int));  
  
// Можем работать с m[i][j]  
  
// Очищать придётся циклом  
for (int i = 0; i < N; ++i)  
    free(m[i]);  
free(m);
```

Двумерный массив на куче: вариант 2

```
int **m = malloc(N * sizeof(int*));  
m[0] = malloc(N * N * sizeof(int));  
for (int i = 1; i < N; ++i)  
    m[i] = m[0] + i * N;  
  
// Можем работать с m[i][j]  
  
free(m[0]);  
free(m);
```


Двумерный массив на куче: вариант 3

```
int *m = malloc(N * N * sizeof(int));
```

```
// Для обращения к элементу (i,j) придётся явно вычислить его адрес
```

```
// Обычно это быстрее, чем прочитать адрес строки m[i] из памяти
```

```
m[i * N + j] = 42;
```

```
free(m);
```

- `calloc` – выделить память и заполнить нулями (нулевыми байтами)

Динамическая память (куча)

- `calloc` – выделить память и заполнить нулями (нулевыми байтами)
- `realloc` – *попробовать* увеличить размер уже выделенного куска

Динамическая память (куча)

- `calloc` – выделить память и заполнить нулями (нулевыми байтами)
- `realloc` – *попробовать* увеличить размер уже выделенного куска
 - Если после него есть свободная память, увеличит размер и вернёт тот же указатель

Динамическая память (куча)

- `calloc` – выделить память и заполнить нулями (нулевыми байтами)
- `realloc` – *попробовать* увеличить размер уже выделенного куска
 - Если после него есть свободная память, увеличит размер и вернёт тот же указатель
 - Если после него нет свободной памяти, выделит новый кусок нужного размера, скопирует туда старые данные, и освободит старый кусок

- Часто хочется объединить несколько переменных в одну логическую единицу:

- Часто хочется объединить несколько переменных в одну логическую единицу:
 - Точка в пространстве: 3 числа `float x, y, z;`

- Часто хочется объединить несколько переменных в одну логическую единицу:
 - Точка в пространстве: 3 числа `float x, y, z;`
 - Ребро взвешенного графа: `int start, end; float weight;`

- Часто хочется объединить несколько переменных в одну логическую единицу:
 - Точка в пространстве: 3 числа `float x, y, z;`
 - Ребро взвешенного графа: `int start, end; float weight;`
 - Вершина бинарного дерева: `node *left, *right; int data;`

- Часто хочется объединить несколько переменных в одну логическую единицу:
 - Точка в пространстве: 3 числа `float x, y, z;`
 - Ребро взвешенного графа: `int start, end; float weight;`
 - Вершина бинарного дерева: `node *left, *right; int data;`
 - Состояние тайла 2D игры: `int terrain; bool has_road;`

- Часто хочется объединить несколько переменных в одну логическую единицу:
 - Точка в пространстве: 3 числа `float x, y, z;`
 - Ребро взвешенного графа: `int start, end; float weight;`
 - Вершина бинарного дерева: `node *left, *right; int data;`
 - Состояние тайла 2D игры: `int terrain; bool has_road;`
- В языке C для этого есть встроенный механизм: *структуры*

```
struct point_3d {  
    float x, y, z;  
};
```

```
struct edge {  
    int start, end;  
    float weight;  
};
```

```
struct node {  
    struct node *left;  
    struct node *right;  
    int data;  
}
```

```
struct tile {  
    int terrain;  
    bool has_road;  
};
```

Структуры

```
// Объявить переменную типа структуры
struct edge e;

// Прочитать поля структуры из файла
scanf("%d %d %f", &e.start, &e.end, &e.weight);

// Завести массив структур
struct edge e[10];
e[4].start = 15;

// Выделить динамический массив структур
struct edge *e = malloc(100 * sizeof(struct edge));

// Обращение к полю через указатель
// (эквивалентные способы)
e[0].weight = 1.412f;
(*e).weight = 3.1415f;
e->weight = 2.71828f;
```

Структуры: инициализация

```
// Инициализация:  
struct edge e = { 10, 15, 1.2345f };  
  
// Designated инициализация:  
struct edge e = {  
    .start = 10,  
    .end = 15,  
    .weight = 1.2345f,  
};
```

Структуры: копирование

```
struct edge e1 = { 10, 15, 1.2345f };  
  
// Полностью копирует все поля  
struct edge e2 = e1;  
  
// Тоже копирует все поля  
e1 = e2;  
  
e1.weight *= 2.f;  
  
// Выведет разные значения  
printf("%f %f", e1.weight, e2.weight);
```

Структуры: копирование

- Если в структуре хранится указатель, то скопируется само значение указателя, а не данные, на которые он указывает

```
struct array {  
    int *data;  
    int size;  
};  
  
struct array a = {  
    .data = malloc(100 * sizeof(int)),  
    .size = 100,  
};  
  
// b.data указывает на ту же память!  
struct array b = a;  
  
b.data[0] = 42;  
printf("%i", a.data[0]); // выведет 42
```


- Как и все объекты, структуры в С можно хранить на стеке (локальная переменная), в глобальной памяти (глобальная/статическая переменная), или на куче (выделена через `malloc`)

- Как и все объекты, структуры в С можно хранить на стеке (локальная переменная), в глобальной памяти (глобальная/статическая переменная), или на куче (выделена через `malloc`)
- Во многих других языках (Java, C#, Python) составные объекты могут храниться только в куче

```
struct big_struct {  
    char name[1024];  
    int values[256];  
};  
  
struct big_struct foo(struct big_struct value) {  
    // ...  
}
```

```
struct big_struct {  
    char name[1024];  
    int values[256];  
};  
  
struct big_struct foo(struct big_struct value) {  
    // ...  
}
```

- Для передачи структуры в функцию придётся выделить на стеке место и скопировать туда аргумент функции

Структуры и функции

```
struct big_struct {  
    char name[1024];  
    int values[256];  
};  
  
struct big_struct foo(struct big_struct value) {  
    // ...  
}
```

- Для передачи структуры в функцию придётся выделить на стеке место и скопировать туда аргумент функции
- Для возвращения структуры из функции придётся сделать то же самое

- В таких случаях лучше передавать указатель на структуру:

```
struct big_struct {  
    char name[1024];  
    int values[256];  
};  
  
void foo(struct big_struct *ptr) {  
    // ...  
}
```

Структуры: typedef

- Писать каждый раз `struct my_struct` надоедает :)

Структуры: typedef

- Писать каждый раз `struct my_struct` надоедает :)
- Вместо этого можно сделать `typedef` – синоним типа для структуры

```
// Новое имя для типа int
typedef int vertex_id;

// Новое имя для типа указателя на функцию
typedef int (*my_func_type)(int, int);

struct my_struct_t {
    char name[64];
    float weight;
};

// Новое имя для типа структуры
typedef struct my_struct my_struct;

// Теперь можно не писать struct!
my_struct x = { "Hello", 6.7f };
```


Структуры: typedef

- **typedef** и объявление структуры можно объединить в одно объявление:

```
typedef struct my_struct {  
    char name[64];  
    float weight;  
} my_struct;
```

```
// Или даже так:  
typedef struct {  
    char name[64];  
    float weight;  
} my_struct;
```

- Не стоит называть структуры в духе `my_struct_t`: стандарт POSIX резервирует все имена, заканчивающиеся на `_t` – можем случайно пересечься с именами под Linux/macOS

Структуры: typedef

- Не стоит называть структуры в духе `my_struct_t`: стандарт POSIX резервирует все имена, заканчивающиеся на `_t` – можем случайно пересечься с именами под Linux/macOS
- Чем `typedef` лучше, чем директива препроцессора `#define`?

- Не стоит называть структуры в духе `my_struct_t`: стандарт POSIX резервирует все имена, заканчивающиеся на `_t` – можем случайно пересечься с именами под Linux/macOS
- Чем `typedef` лучше, чем директива препроцессора `#define`? `#define` применяется ко всем идентификаторам, а `typedef` – только к *типам*

```
struct test {  
    int i;  
    char c;  
};
```

```
struct test {  
    int i;  
    char c;  
};
```

- Чему будет равно `sizeof(struct test)`?

```
struct test {  
    int i;  
    char c;  
};
```

- Чему будет равно `sizeof(struct test)`? На большинстве платформ – 8:
 - 4 байта на `int i`
 - 1 байт на `char c`
 - 3 байта на *padding*

- Компилятор может сгенерировать более эффективный код, если адрес объекта типа `int` кратен 4 байтам (“выровнен” на 4 байта)

- Компилятор может сгенерировать более эффективный код, если адрес объекта типа `int` кратен 4 байтам (“выровнен” на 4 байта)
 - Обычно, выравнивание для примитивного типа равно его размеру

- Компилятор может сгенерировать более эффективный код, если адрес объекта типа `int` кратен 4 байтам (“выровнен” на 4 байта)
 - Обычно, выравнивание для примитивного типа равно его размеру
- $\implies$ По умолчанию компилятор вставляет дополнительные неиспользуемые байты (*padding*), чтобы обеспечить это